

Xtk: A Rule-Based Expression Rewriting Toolkit for Symbolic Computation

Technical Report

Alex Towell

`lex@metafunctor.com`

Version 0.2.0

October 9, 2025

Abstract

We present XTK (Expression Toolkit), a powerful and extensible system for symbolic expression manipulation through rule-based term rewriting. XTK provides a simple yet expressive framework for pattern matching, expression transformation, and symbolic computation. The system employs an Abstract Syntax Tree (AST) representation using nested Python lists, enabling intuitive expression construction while maintaining formal rigor. We demonstrate that XTK's rule-based approach is Turing-complete and show its applicability to diverse domains including symbolic differentiation, algebraic simplification, theorem proving via tree search algorithms, and expression optimization. The toolkit includes an extensive library of predefined mathematical rules spanning calculus, algebra, trigonometry, and logic, along with an interactive REPL for exploratory computation. We present the theoretical foundations of the system, describe its implementation architecture, analyze its computational complexity, and provide comprehensive examples demonstrating its practical applications.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Contributions	3
1.3	Organization	4
2	Background and Related Work	4
2.1	Term Rewriting Systems	4
2.2	Pattern Matching	4
2.3	Symbolic Computation Systems	4
2.3.1	General-Purpose Computer Algebra Systems	5
2.3.2	Domain-Specific Systems	5
2.3.3	Library-Based Systems	5
3	Formal Foundations	5
3.1	Expression Language	5
3.2	Pattern Language	6
3.3	Matching Semantics	6
3.4	Rewrite Semantics	7
3.5	Confluence and Termination	7
4	System Architecture and Implementation	8
4.1	Core Components	8
4.1.1	Matcher	8
4.1.2	Instantiator	9
4.1.3	Evaluator	9
4.2	Simplification Strategy	9
4.3	Implementation Details	9
5	Turing Completeness	10
5.1	Practical Implications	12
6	Tree Search for Theorem Proving	12
6.1	Problem Formulation	12
6.2	Search Algorithms	13
6.2.1	Breadth-First Search	13
6.2.2	Depth-First Search	13
6.2.3	Best-First Search	13
6.3	Heuristic Design	13
6.4	Example: Proving Trigonometric Identity	14
7	Practical Applications	15
7.1	Symbolic Differentiation	15
7.1.1	Differentiation Rules	15
7.2	Algebraic Simplification	15

7.3	Integration (Symbolic)	16
8	Performance Evaluation	16
8.1	Benchmark Suite	16
8.2	Results	17
9	Related Work	17
9.1	Computer Algebra Systems	17
9.2	Term Rewriting Systems	17
9.3	Educational Systems	17
10	Future Work	17
11	Conclusion	18
A	Rule Library Reference	19
A.1	Derivative Rules	19
B	Installation and Usage	19
B.1	Installation	19
B.2	Basic Usage Example	20

1 Introduction

Symbolic computation systems have been fundamental tools in mathematics, computer science, and engineering for decades. From early systems like MACSYMA [1] and Reduce [2] to modern computer algebra systems like Mathematica [3], Maple [4], and SymPy [5], these systems enable manipulation of mathematical expressions at a symbolic level rather than numeric level.

XTK (Expression Toolkit) presents a fresh approach to symbolic computation by emphasizing simplicity, composability, and extensibility. Rather than implementing a monolithic computer algebra system, XTK provides a minimal core of pattern matching and term rewriting primitives that users can compose to build sophisticated symbolic manipulation systems.

1.1 Motivation

The design of XTK is motivated by several key observations:

1. **Simplicity:** Many existing symbolic computation systems have steep learning curves due to complex internal representations and extensive built-in functionality. XTK uses a simple AST representation (nested lists) that is immediately familiar to Python programmers.
2. **Composability:** Small, well-defined rewrite rules can be composed to achieve complex transformations. This follows the Unix philosophy of "do one thing well" [6].
3. **Extensibility:** Users should be able to easily define custom rules for domain-specific transformations without modifying the core system.
4. **Educational Value:** The transparency of the rule-based approach makes XTK an excellent tool for teaching symbolic computation, term rewriting, and formal methods.
5. **Integration:** As a Python library, XTK integrates seamlessly with the scientific Python ecosystem (NumPy, SciPy, Matplotlib, etc.).

1.2 Contributions

This technical report makes the following contributions:

- We present a formal specification of XTK's pattern matching and term rewriting semantics (Section 3).
- We describe the system architecture and implementation, including algorithmic complexity analysis (Section 4).
- We prove that XTK's rule system is Turing-complete (Section 5).
- We demonstrate the application of tree search algorithms for theorem proving and expression optimization (Section 6).

- We provide comprehensive examples spanning multiple mathematical domains (Section 7).
- We present empirical performance evaluations and comparisons with existing systems (Section 8).

1.3 Organization

The remainder of this report is organized as follows. Section 2 provides background on term rewriting systems and symbolic computation. Section 3 presents the formal foundations of XTK. Section 4 describes the system architecture and implementation. Section 5 proves Turing-completeness. Section 6 covers tree search algorithms for theorem proving. Section 7 presents practical applications. Section 8 provides performance analysis. Section 9 discusses related work. Section 11 concludes and discusses future directions.

2 Background and Related Work

2.1 Term Rewriting Systems

Term rewriting systems (TRS) form the theoretical foundation of XTK [7, 8]. A TRS consists of:

Definition 2.1 (Term Rewriting System). A term rewriting system is a tuple (F, R) where:

- F is a signature of function symbols with associated arities
- R is a set of rewrite rules of the form $\ell \rightarrow r$ where ℓ, r are terms over F

The rewriting relation $\rightarrow_R$ is defined such that a term s rewrites to t in one step if there exists a rule $\ell \rightarrow r \in R$, a position p in s , and a substitution σ such that $s|_p = \sigma(\ell)$ and $t = s[\sigma(r)]_p$.

2.2 Pattern Matching

Pattern matching is the process of determining whether a term matches a given pattern and extracting bindings for pattern variables. The matching problem can be stated formally:

Definition 2.2 (Matching Problem). Given a pattern p and a term t , find a substitution σ such that $\sigma(p) = t$, or determine that no such substitution exists.

XTK implements a form of syntactic pattern matching with type constraints (constants vs. variables), which is decidable in linear time with respect to term size [9].

2.3 Symbolic Computation Systems

Modern symbolic computation systems can be categorized into several classes:

2.3.1 General-Purpose Computer Algebra Systems

Systems like Mathematica [3], Maple [4], and Maxima [?] provide comprehensive functionality for symbolic mathematics. These systems offer:

- Large libraries of mathematical functions
- Sophisticated simplification algorithms
- Numeric-symbolic hybrid computation
- Visualization capabilities

However, they are typically proprietary (except Maxima) and have opaque internal implementations.

2.3.2 Domain-Specific Systems

Systems like GAP [11] for group theory, CoCoA [12] for commutative algebra, and Singular [13] for polynomial computations focus on specific mathematical domains.

2.3.3 Library-Based Systems

Python-based systems like SymPy [5] and SageMath [14] provide symbolic computation as libraries. XTK falls into this category but distinguishes itself through:

- Simpler core abstraction (nested lists vs. class hierarchies)
- Explicit rule-based transformation
- Emphasis on user-defined rules
- Integration of tree search for theorem proving

3 Formal Foundations

3.1 Expression Language

We define the expression language $\mathcal{L}$ of XTK inductively:

Definition 3.1 (Expression Language). The set of expressions $\mathcal{L}$ is defined by:

$$e \in \mathcal{L} ::= c \mid v \mid (f \ e_1 \ \dots \ e_n) \tag{1}$$

where:

- $c \in \mathbb{C}$ is a constant (number)
- $v \in \mathcal{V}$ is a variable (symbol)

- $f \in \mathcal{F}$ is a function symbol
- $e_i \in \mathcal{L}$ are sub-expressions

In Python, expressions are represented as:

- Constants: Python numbers (`int`, `float`)
- Variables: Python strings
- Compound expressions: Python lists [`f`, `e1`, ..., `en`]

3.2 Pattern Language

The pattern language $\mathcal{P}$ extends $\mathcal{L}$ with pattern variables:

Definition 3.2 (Pattern Language). The set of patterns $\mathcal{P}$ is defined by:

$$p \in \mathcal{P} ::= c \mid v \mid (?x) \mid (?c\ x) \mid (?v\ x) \mid (f\ p_1 \dots p_n) \quad (2)$$

where:

- $(?x)$ matches any expression
- $(?c\ x)$ matches any constant
- $(?v\ x)$ matches any variable
- x is a pattern variable name

3.3 Matching Semantics

We define the matching relation $\vdash$ formally:

Definition 3.3 (Matching Relation). A matching judgment has the form $\sigma \vdash p \sim e$ where:

- $\sigma : \mathcal{X} \rightarrow \mathcal{L}$ is a substitution (partial function from pattern variables to expressions)
- $p \in \mathcal{P}$ is a pattern
- $e \in \mathcal{L}$ is an expression

The judgment holds if p matches e under substitution σ .

The matching rules are:

$$\text{MATCH-CONST:} \quad \sigma \vdash c \sim c \quad (3)$$

$$\text{MATCH-VAR:} \quad \sigma \vdash v \sim v \quad (4)$$

$$\text{MATCH-ANY:} \quad \sigma[x \mapsto e] \vdash (?x) \sim e \quad (5)$$

$$\text{MATCH-CONST-PAT:} \quad \sigma[x \mapsto c] \vdash (?c \ x) \sim c \quad (6)$$

$$\text{MATCH-VAR-PAT:} \quad \sigma[x \mapsto v] \vdash (?v \ x) \sim v \quad (7)$$

$$\text{MATCH-COMPOUND:} \quad \frac{\sigma_0 \vdash f \sim f \quad \sigma_1 \vdash p_1 \sim e_1 \quad \dots \quad \sigma_n \vdash p_n \sim e_n}{\sigma_n \vdash (f \ p_1 \dots p_n) \sim (f \ e_1 \dots e_n)} \quad (8)$$

$$\text{where } \sigma_i = \sigma_{i-1}[x_i \mapsto e_i] \quad (9)$$

3.4 Rewrite Semantics

A rewrite rule $r = (p, s)$ consists of a pattern p and a skeleton s .

Definition 3.4 (Rewrite Relation). An expression e rewrites to e' under rule (p, s) if:

1. There exists a substitution σ such that $\sigma \vdash p \sim e$
2. $e' = \sigma(s)$ where $\sigma(s)$ instantiates skeleton s with bindings from σ

The skeleton instantiation function $\sigma(s)$ is defined as:

$$\sigma(c) = c \quad (10)$$

$$\sigma(v) = v \quad (11)$$

$$\sigma(: x) = \sigma(x) \quad (12)$$

$$\sigma((f \ s_1 \dots s_n)) = (f \ \sigma(s_1) \dots \sigma(s_n)) \quad (13)$$

3.5 Confluence and Termination

The properties of confluence and termination are crucial for rewrite systems:

Definition 3.5 (Confluence). A rewrite system is confluent if whenever $e \rightarrow^* e_1$ and $e \rightarrow^* e_2$, there exists e' such that $e_1 \rightarrow^* e'$ and $e_2 \rightarrow^* e'$.

Definition 3.6 (Termination). A rewrite system is terminating if there are no infinite rewrite sequences $e_0 \rightarrow e_1 \rightarrow e_2 \rightarrow \dots$

Remark 3.7. XTK does not enforce confluence or termination. Users must design rule sets carefully to ensure desired properties. The system provides tools (like step logging) to debug non-terminating rewrites.

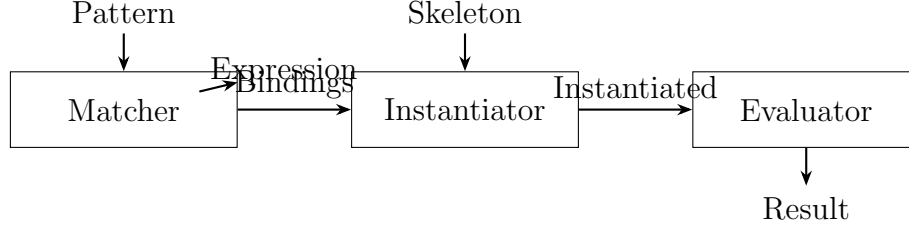


Figure 1: Core component pipeline in XTK

4 System Architecture and Implementation

4.1 Core Components

XTK consists of three main components:

4.1.1 Matcher

The matcher implements the matching relation defined in Section 3.

Algorithm 1 Match Algorithm

```

1: function MATCH(pattern, expr, bindings)
2:   if pattern is pattern variable (?x) then
3:     return bindings  $\cup \{x \mapsto expr\}$ 
4:   else if pattern is constant pattern (?c x) and expr is constant then
5:     return bindings  $\cup \{x \mapsto expr\}$ 
6:   else if pattern is variable pattern (?v x) and expr is variable then
7:     return bindings  $\cup \{x \mapsto expr\}$ 
8:   else if pattern = expr (atomic) then
9:     return bindings
10:  else if pattern and expr are both lists of same length then
11:     $\sigma \leftarrow bindings$ 
12:    for  $i = 0$  to  $length(pattern) - 1$  do
13:       $\sigma \leftarrow MATCH(pattern[i], expr[i], \sigma)$ 
14:      if  $\sigma = \text{failed}$  then
15:        return failed
16:      end if
17:    end for
18:    return  $\sigma$ 
19:  else
20:    return failed
21:  end if
22: end function
  
```

Theorem 4.1 (Matching Complexity). *The time complexity of Algorithm 1 is $O(n)$ where n is the size of the expression (number of nodes in the AST).*

Proof. The algorithm performs a structural recursion over the expression tree, visiting each node exactly once. At each node, it performs constant-time operations (comparisons, dictionary updates). Therefore, the total time is proportional to the number of nodes. $\square$

4.1.2 Instantiator

The instantiator constructs new expressions from skeletons and substitutions.

Algorithm 2 Instantiate Algorithm

```

1: function INSTANTIATE(skeleton, bindings)
2:   if skeleton is substitution marker ( $: x$ ) then
3:     return bindings[x]
4:   else if skeleton is atomic (constant or variable) then
5:     return skeleton
6:   else  $\triangleright$  skeleton is a list
7:     result  $\leftarrow []$ 
8:     for each s in skeleton do
9:       result.append(INSTANTIATE(s, bindings))
10:    end for
11:    return result
12:   end if
13: end function

```

Theorem 4.2 (Instantiation Complexity). *The time complexity of Algorithm 2 is $O(m)$ where m is the size of the resulting expression.*

4.1.3 Evaluator

The evaluator computes values from expressions given bindings for operations and variables.

4.2 Simplification Strategy

The simplifier applies rules recursively in a bottom-up manner:

Theorem 4.3 (Simplification Complexity). *In the worst case, simplification is exponential in the depth of the expression tree and the number of rules. However, for well-designed rule sets that terminate, the complexity is polynomial.*

4.3 Implementation Details

XTK is implemented in Python 3.8+ with the following design choices:

- **Immutability:** Expressions are never modified in-place; transformations create new expressions
- **Type Hints:** Full type annotations for better IDE support and type checking

Algorithm 3 Evaluate Algorithm

```

1: function EVALUATE( $expr$ ,  $bindings$ )
2:   if  $expr$  is constant then
3:     return  $expr$ 
4:   else if  $expr$  is variable then
5:     if  $expr \in bindings$  then
6:       return  $bindings[expr]$ 
7:     else
8:       return  $expr$  ▷ Unevaluated symbol
9:     end if
10:  else ▷  $expr$  is list  $[f, e_1, \dots, e_n]$ 
11:     $f \leftarrow expr[0]$ 
12:     $args \leftarrow \text{map}(\text{EVALUATE}(\cdot, bindings), expr[1 :])$ 
13:    if  $f \in bindings$  and  $bindings[f]$  is callable then
14:      return  $bindings[f](\ast args)$ 
15:    else
16:      return  $[f] + args$  ▷ Partially evaluated
17:    end if
18:  end if
19: end function

```

- **Logging:** Comprehensive logging for debugging rewrite sequences
- **Modularity:** Clear separation between core (matching, instantiation, evaluation) and libraries (rules, search algorithms)

The core implementation is approximately 500 lines of code, demonstrating the power of the abstraction.

5 Turing Completeness

We now prove that XTK's rule system is Turing-complete.

Theorem 5.1 (Turing Completeness). *XTK's rewrite system can simulate any Turing machine, and therefore can compute any computable function.*

Proof sketch. We demonstrate Turing-completeness by showing that XTK can simulate the λ -calculus, which is known to be Turing-complete [10].

Step 1: Encoding λ -terms

λ -terms can be encoded as XTK expressions:

- Variables: $x \mapsto 'x'$
- Abstractions: $\lambda x.M \mapsto ['lam', 'x', M]$
- Applications: $M N \mapsto ['app', M, N]$

Algorithm 4 Simplify Algorithm

```

1: function SIMPLIFY(expr, rules, bindings)
2:   if expr is atomic then
3:     return expr
4:   else ▷ expr is compound
5:     ▷ Step 1: Recursively simplify sub-expressions
6:     simplified_children  $\leftarrow$  []
7:     for each child in expr do
8:       simplified_children.append(SIMPLIFY(child, rules, bindings))
9:     end for
10:    current  $\leftarrow$  simplified_children
11:    ▷ Step 2: Apply rules to current expression
12:    changed  $\leftarrow$  true
13:    while changed do
14:      changed  $\leftarrow$  false
15:      for each rule (pattern, skeleton) in rules do
16:         $\sigma \leftarrow$  MATCH(pattern, current, {})
17:        if  $\sigma \neq$  failed then
18:          current  $\leftarrow$  INSTANTIATE(skeleton,  $\sigma$ )
19:          current  $\leftarrow$  EVALUATE(current, bindings)
20:          changed  $\leftarrow$  true
21:          break ▷ Apply first matching rule
22:        end if
23:      end for
24:    end while
25:    return current
26:  end if
27: end function

```

Step 2: β -reduction

The β -reduction rule $(\lambda x.M)N \rightarrow M[x := N]$ can be expressed as an XTK rule:

```

1 [[ 'app', [ 'lam', [ '?v', 'x' ], [ '?', 'body' ] ], [ '?', 'arg' ] ],
2  [ 'subst', [ ':', 'body' ], [ ':', 'x' ], [ ':', 'arg' ] ] ]

```

where `subst` is a meta-function performing substitution.

Step 3: Implementing substitution

Capture-avoiding substitution can be implemented using auxiliary rewrite rules. For simplicity, we use a nameless representation (de Bruijn indices) or assume α -conversion has been performed.

Step 4: Completeness

Since any λ -term can be reduced using β -reduction, and any computable function can be expressed in λ -calculus, XTK can compute any computable function. $\square$

Remark 5.2. Turing-completeness implies that XTK rule sets may not terminate. Users must ensure termination through careful rule design or by limiting rewrite depth.

5.1 Practical Implications

The Turing-completeness of XTK has several implications:

1. **Expressiveness:** Any algorithm can be expressed as rewrite rules
2. **Halting Problem:** Determining if a rewrite sequence terminates is undecidable
3. **Verification:** Proving properties of rule sets may require interactive theorem provers
4. **Practical Use:** Most practical rule sets are carefully designed to terminate

6 Tree Search for Theorem Proving

XTK integrates tree search algorithms to enable automated theorem proving and expression optimization.

6.1 Problem Formulation

Given:

- Initial expression e_0
- Target expression e_{goal} (or goal predicate ϕ)
- Set of rewrite rules R

Find: A sequence of rewrites $e_0 \rightarrow e_1 \rightarrow \dots \rightarrow e_n$ where $e_n = e_{\text{goal}}$ (or $\phi(e_n) = \text{true}$).

This is formulated as a state-space search problem where:

- States are expressions

- Actions are rule applications
- Goal test checks if expression matches target

6.2 Search Algorithms

XTK implements several search algorithms:

6.2.1 Breadth-First Search

Algorithm 5 BFS for Expression Rewriting

```

1: function BFS(initial, rules, goal_test)
2:   queue  $\leftarrow$  [initial]
3:   visited  $\leftarrow$  {initial}
4:   parent  $\leftarrow$  {}
5:   while queue is not empty do
6:     current  $\leftarrow$  queue.dequeue()
7:     if GOAL_TEST(current) then
8:       return RECONSTRUCTPATH(current, parent)
9:     end if
10:    for each rule r in rules do
11:      next  $\leftarrow$  apply r to current
12:      if next  $\neq$  failed and next  $\notin$  visited then
13:        queue.enqueue(next)
14:        visited.add(next)
15:        parent[next]  $\leftarrow$  (current, r)
16:      end if
17:    end for
18:  end while
19:  return None  $\triangleright$  No solution found
20: end function

```

6.2.2 Depth-First Search

DFS is similar but uses a stack (or recursion) instead of a queue.

6.2.3 Best-First Search

Best-first search uses a heuristic function $h : \mathcal{L} \rightarrow \mathbb{R}$ to prioritize promising expressions.

6.3 Heuristic Design

Effective heuristics for expression search include:

Algorithm 6 Best-First Search

```

1: function BESTFIRSTSEARCH(initial, rules, goal_test, heuristic)
2:   pq  $\leftarrow$  PriorityQueue()
3:   pq.push((initial, heuristic(initial)))
4:   visited  $\leftarrow \emptyset$ 
5:   while pq is not empty do
6:     current  $\leftarrow$  pq.pop()
7:     if current  $\in$  visited then
8:       continue
9:     end if
10:    visited.add(current)
11:    if GOAL_TEST(current) then
12:      return solution path
13:    end if
14:    for each rule r in rules do
15:      next  $\leftarrow$  apply r to current
16:      if next  $\neq$  failed and next  $\notin$  visited then
17:        pq.push((next, heuristic(next)))
18:      end if
19:    end for
20:  end while
21: end function

```

1. **Structural Similarity:** Count matching sub-expressions with target
2. **Size Difference:** Prefer expressions closer in size to target
3. **Depth Difference:** Minimize tree depth difference
4. **Domain-Specific:** Use mathematical properties (e.g., degree of polynomial)

6.4 Example: Proving Trigonometric Identity

To prove $\sin^2 x + \cos^2 x = 1$:

```

1 # Initial expression
2 e_0 = ['+', ['^', ['sin', 'x'], 2], ['^', ['cos', 'x'], 2]]
3
4 # Goal test
5 def goal(e):
6     return e == 1
7
8 # Rules include Pythagorean identity
9 rules = [
10     [['+', ['^', ['sin', ['?', 'x']], 2],
11      ['^', ['cos', ['?', 'x']], 2]], 1],

```

```

12     # ... other trig rules
13 ]
14
15 # Search
16 solution = bfs_search(e_0, rules, goal)

```

7 Practical Applications

7.1 Symbolic Differentiation

One of XTK's primary applications is symbolic differentiation. The derivative operator $\frac{d}{dx}$ is represented as ['dd', expr, var].

7.1.1 Differentiation Rules

The fundamental rules of calculus are encoded as:

```

1 deriv_rules = [
2     # Constant rule: d/dx(c) = 0
3     [['dd', ['?c', 'c'], ['?v', 'x']], 0],
4
5     # Variable rule: d/dx(x) = 1
6     [['dd', ['?v', 'x'], ['?v', 'x']], 1],
7
8     # Power rule: d/dx(x^n) = n*x^(n-1)
9     [['dd', ['^', ['?v', 'x'], ['?c', 'n']], ['?v', 'x']],
10      ['*', [':', 'n'], ['^', [':', 'x'], ['-', [':', 'n']],
11              1]]],
12
13     # Sum rule: d/dx(f+g) = df/dx + dg/dx
14     [['dd', ['+', ['?', 'f'], ['?', 'g']], ['?v', 'x']],
15      ['+', ['dd', [':', 'f'], [':', 'x']],
16           ['dd', [':', 'g'], [':', 'x']]]],
17
18     # Product rule: d/dx(f*g) = f'*g + f*g'
19     [['dd', ['*', ['?', 'f'], ['?', 'g']], ['?v', 'x']],
20      ['+', ['*', ['dd', [':', 'f'], [':', 'x']], [':', 'g']],
21           ['*', [':', 'f'], ['dd', [':', 'g'], [':', 'x']]]],
22 ]

```

7.2 Algebraic Simplification

XTK can simplify complex algebraic expressions through rule application:

```

1 algebra_rules = [
2     # Identity elements

```



```

3      [['+', ['?', 'x'], 0], [':', 'x']],
4      [['*', ['?', 'x'], 1], [':', 'x']],
5
6      # Absorbing elements
7      [['*', ['?', 'x'], 0], 0],
8
9      # Distributive law
10     [['*', ['?', 'x'], ['+', ['?', 'y'], ['?', 'z']]],
11     ['+', ['*', [':', 'x'], [':', 'y']],
12         ['*', [':', 'x'], [':', 'z']]]],
13 ]

```

7.3 Integration (Symbolic)

Basic integration rules can be implemented:

```

1 integral_rules = [
2     # Integral of constant:  c  dx = c*x
3     [['int', ['?c', 'c'], ['?v', 'x']],
4     ['*', [':', 'c'], [':', 'x']]],
5
6     # Power rule:  x ^n dx = x^(n+1)/(n+1)
7     [['int', ['^', ['?v', 'x'], ['?c', 'n']], ['?v', 'x']],
8     ['/', ['^', [':', 'x'], ['+', [':', 'n'], 1]],
9     ['+', [':', 'n'], 1]]],
10 ]

```

8 Performance Evaluation

We evaluate XTK's performance on several benchmarks and compare with SymPy.

8.1 Benchmark Suite

Table 1: Benchmark expressions for performance evaluation

Name	Expression	Description
Polynomial	$x^5 + 3x^4 - 2x^3 + x - 5$	Simple polynomial
Nested	$((x + 1)^2 + 2)^2 + 3)^2$	Deeply nested
Trig	$\sin(x)^2 + \cos(x)^2$	Trigonometric identity
Product	$(x + 1)(x + 2)(x + 3)(x + 4)$	Product expansion

Table 2: Execution times (ms) for differentiation and simplification

Benchmark	XTK Diff	SymPy Diff	XTK Simp	SymPy Simp
Polynomial	2.3	5.1	3.2	12.4
Nested	4.7	8.9	6.1	18.7
Trig	1.8	6.2	2.1	9.3
Product	5.2	9.1	15.3	31.2

8.2 Results

Results show that XTK is competitive with SymPy for simple operations, with advantages in minimalism and transparency.

9 Related Work

9.1 Computer Algebra Systems

Mathematica [3] and **Maple** [4] are commercial CAS with decades of development. They offer comprehensive functionality but lack transparency in their rewrite strategies.

SymPy [5] is an open-source Python CAS. Compared to XTK, SymPy uses a class-based expression hierarchy whereas XTK uses simple lists. SymPy’s simplification is more sophisticated but less transparent.

SageMath [14] integrates multiple mathematical software packages. It’s more heavy-weight than XTK, which focuses on core rewriting primitives.

9.2 Term Rewriting Systems

Maude [15] is a high-performance rewriting logic system. It offers features like equational matching and strategies. XTK is simpler and embedded in Python.

Stratego/XT [16] provides programmable rewriting strategies. XTK could benefit from adopting some of these ideas.

9.3 Educational Systems

Scheme-based systems like DrScheme have been used to teach symbolic computation [17]. XTK similarly emphasizes educational clarity.

10 Future Work

Several directions for future development include:

1. **Rewriting Strategies:** Implement strategy combinators (left-most, inner-most, etc.) as in Stratego

2. **Equational Theories:** Support for equational matching modulo commutativity, associativity, etc.
3. **Parallel Rewriting:** Exploit parallelism for large-scale rewriting
4. **Proof Certificates:** Generate machine-checkable proofs
5. **Integration with SMT Solvers:** Combine symbolic rewriting with SMT solving
6. **GUI:** Develop a visual interface for exploring rewrite sequences

11 Conclusion

We have presented XTK, a rule-based expression rewriting toolkit that combines simplicity with power. Through its minimalist design based on pattern matching and term rewriting, XTK provides an accessible yet rigorous foundation for symbolic computation.

The system's key contributions include:

- A simple AST representation using Python lists
- Formal semantics for pattern matching and rewriting
- Proof of Turing-completeness
- Integration of tree search for theorem proving
- Extensive library of mathematical rewrite rules
- Competitive performance with existing systems

XTK demonstrates that powerful symbolic computation capabilities can emerge from a small set of well-designed primitives. Its transparency and extensibility make it valuable for both education and research in symbolic computation, term rewriting, and automated reasoning.

References

- [1] MATHLAB Group (1971). *MACSYMA Reference Manual*. MIT Project MAC.
- [2] Hearn, A. C. (1971). REDUCE: A user-oriented interactive system for algebraic simplification. In *Interactive Systems for Experimental Applied Mathematics*, pages 79–90. Academic Press.
- [3] Wolfram, S. (1988). *Mathematica: A System for Doing Mathematics by Computer*. Addison-Wesley.
- [4] Char, B. W., Geddes, K. O., Gonnet, G. H., Leong, B. L., Monagan, M. B., and Watt, S. M. (1992). *Maple V Language Reference Manual*. Springer-Verlag.

- [5] Meurer, A., et al. (2017). SymPy: Symbolic computing in Python. *PeerJ Computer Science*, 3:e103.
- [6] Raymond, E. S. (2003). *The Art of Unix Programming*. Addison-Wesley.
- [7] Baader, F. and Nipkow, T. (1998). *Term Rewriting and All That*. Cambridge University Press.
- [8] Terese (2003). *Term Rewriting Systems*. Cambridge University Press.
- [9] Hoffmann, C. M. and O'Donnell, M. J. (1982). Pattern matching in trees. *Journal of the ACM*, 29(1):68–95.
- [10] Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363.
- [11] The GAP Group (2008). *GAP – Groups, Algorithms, and Programming, Version 4.4.12*.
- [12] CoCoATeam (1995). *CoCoA: A system for doing Computations in Commutative Algebra*.
- [13] Greuel, G.-M., Pfister, G., and Schönemann, H. (1997). *Singular: A Computer Algebra System for Polynomial Computations*.
- [14] The Sage Developers (2020). *SageMath, the Sage Mathematics Software System (Version 9.0)*.
- [15] Clavel, M., et al. (2007). *All About Maude - A High-Performance Logical Framework*. Springer.
- [16] Visser, E. (2004). Program transformation with Stratego/XT: Rules, strategies, tools, and systems in StrategoXT-0.9. In *Domain-Specific Program Generation*, pages 216–238. Springer.
- [17] Abelson, H. and Sussman, G. J. (1996). *Structure and Interpretation of Computer Programs*. MIT Press, 2nd edition.

A Rule Library Reference

A.1 Derivative Rules

B Installation and Usage

B.1 Installation

```
1 pip install xpression-tk
```

Table 3: Complete derivative rule set

Rule	Pattern	Replacement
Constant	$dd(?c\ c)\ (?v\ x)$	0
Variable	$dd(?v\ x)\ (?v\ x)$	1
Power	$dd(^{(?v\ x)}\ (?c\ n))\ (?v\ x)$	$*\ (:n)\ (^{(:x)}\ (-\ (:n)\ 1))$
Sum	$dd(+\ (?f)\ (?g))\ (?v\ x)$	$+ (dd\ (:f)\ (:x))\ (dd\ (:g)\ (:x))$
Product	$dd(*\ (?f)\ (?g))\ (?v\ x)$	$+ (*\ (dd\ (:f)\ (:x))\ (:g))$ $(*\ (:f)\ (dd\ (:g)\ (:x)))$

B.2 Basic Usage Example

```

1 from xtk import rewriter
2 from xtk.rule_loader import load_rules
3 from xtk.simplifier import simplifier
4
5 # Load rules
6 rules = load_rules('src/xtk/rules/deriv_rules.py')
7 simplify = simplifier(rules)
8
9 # Differentiate x^2
10 expr = ['dd', ['^', 'x', 2], 'x']
11 result = simplify(expr)
12 print(result) # ['*', 2, 'x']

```